audita

AGENTIC SMART CONTRACT SECURITY REVIEW · PREPARED FOR

Iceberg

A USDC payout pool for simulated prop trading on Arbitrum

Produced with Audita's AI security tooling

DATE

September 29, 2026

AUDITED COMMIT

`310c7cd`

01 ABOUT

Audita

Audita is a Web3 security company specializing in smart contract security reviews for protocols, decentralized applications, and on-chain infrastructure.

We help teams identify vulnerabilities before they reach production through rigorous manual code review, adversarial security research, architecture analysis, and automated testing.

Our team consists of 50+ senior security researchers with proven experience across DeFi, infrastructure, wallets, trading systems, and complex on-chain applications.

To date, Audita has completed more than 100 security reviews across the Web3 ecosystem, working with teams at different stages ranging from pre-launch protocols to production systems securing significant on-chain value.

Every engagement is approached from both an offensive and defensive perspective: understanding how the system is intended to behave, identifying where those assumptions can break, and determining how an attacker could exploit them.

This report was produced through Audita's AI-assisted security review workflow. Our tooling combines codebase-wide analysis, targeted vulnerability investigation, and reproducible validation to identify and explain security issues within the audited scope.

PREVIOUS WORK	WEBSITE
Audita public reports	audita.io

02 METHODOLOGY

How this review works

Audita's AI-assisted workflow maps the supplied codebase, investigates candidate failure paths, and checks them against the available source and test context. This is automated and agent-driven analysis, not a manual audit by the research team.

Each finding records the observed failure mode, impact, and recommended remediation. This initial report does not evaluate fixes or assign remediation statuses.

Risk classification

Severity reflects both the impact of a successful exploit and the likelihood of reaching it under the stated assumptions.

Likelihood / Impact	High	Medium	Low
High	HIGH	HIGH	MEDIUM
Medium	HIGH	MEDIUM	LOW
Low	MEDIUM	LOW	LOW

Informational findings have no direct security impact and sit outside the matrix.

IMPACT

High	Major asset loss, critical control compromise, or protocol-wide disruption.
Medium	Material but bounded loss or disruption.
Low	Limited loss, disruption, or scope.

LIKELIHOOD

High	Practical under ordinary on-chain conditions.
Medium	Requires specific but plausible conditions.
Low	Requires unusual conditions or disproportionate resources.

DISCLAIMER

AI-assisted analysis can miss vulnerabilities that require human judgment, business context, or adversarial reasoning. It is bounded by time, scope, and available evidence and cannot guarantee the absence of vulnerabilities. A manual audit by experienced researchers is advised before production deployment, major upgrades, or custody of user funds.

03 EXECUTIVE SUMMARY

FINDINGS IDENTIFIED **15**

SEVERITY DISTRIBUTION

2 Medium	5 Low	8 Informational

CONTRACTS IN SCOPE

contracts/src/IcebergPayoutVault.sol	contracts/src/IcebergPayoutVaultV2.sol
contracts/src/IcebergPayoutVaultV3.sol	contracts/src/IcebergPayoutVaultV4.sol

Review window: September 29, 2026 – September 29, 2026 · 1 days

AUDITED COMMIT HASH `310c7cd0c6e367edc87cf579283f9f35446afc42`

04 PROJECT SUMMARY

What Iceberg is

Iceberg lets traders pay USDC for instant access to simulated trading accounts and claim 80% of eligible profits from a USDC payout pool.

The reviewed vault revisions use a Safe budget and two independent EIP-712 signers to authorize payments and later share releases. V3 and V4 separate distributable revenue from an insurance target, reserve team and depositor shares, and carry obligations into successor vaults.

The review followed accounting and migration paths into the off-chain services where required. It found two defects that disrupt revenue and post-migration processing, five lower-impact availability or accounting weaknesses, and eight operational or observability limitations.

FINDINGS OVERVIEW

AT AUDITED COMMIT `310c7cd`

Accounting and service coordination

`M-01 · M-02 · L-01 · I-07`

On-chain accounting mixed L1 block numbers, recognized revenue, insurance backing, and spare capital with assumptions made by off-chain services. Routine refunds and capital movements could suppress later commission revenue, while revenue recognition remained available during a pause.

Share lifecycle

`L-02 · L-03 · L-04 · I-01 · I-04`

Shares bound delivery to recorded recipients without a cancellation or rebinding path. Delivery ordering, wallet rotation, credit identity gaps, and native USDC controls could delay or permanently strand individual obligations.

Migration and governance recovery

`L-05 · I-02 · I-03 · I-05 · I-06 · I-08`

Migration and governance paths favored immutable state and one-way transitions. Recovery depended on Safe action, copied state was only partly visible in events, and predecessor reads became more expensive with every successor.

05 CONTRACTS & ROLES

Core contracts

CONTRACT	ROLE
`contracts/src/IcebergPayoutVault.sol`	Pays 80% of eligible profits and reserves team and referral shares under a Safe budget and two independent signatures.
`contracts/src/IcebergPayoutVaultV2.sol`	Retains the team remainder as reusable capital while preserving the V1 budget, signer, and delivery controls.
`contracts/src/IcebergPayoutVaultV3.sol`	Adds insurance and revenue accounting, depositor shares, checkout refunds, treasury withdrawals, and vault migration.
`contracts/src/IcebergPayoutVaultV4.sol`	Adds a fixed manager commission and carries V3 or V4 history through a recursive predecessor chain.

The review covered the four payout vault revisions and their linked migration, revenue, and refund behavior.

PRIVILEGED ROLES

Payout Safe. Two of four owners govern V1 and V2, while three of four owners govern V3 and V4, with control over budgets, pause state, signers, recipients, recovery, refunds, and migration.

Entitlement signer. Held by the isolated entitlement service, it jointly authorizes payments, share releases, and revenue recognition and can pause the vault.

Risk signer. Held by the isolated risk service, it independently authorizes payments, share releases, and revenue recognition and can pause the vault.

06 FINDINGS

What we found

ID	TITLE	SEVERITY
M-01	The vault records the L1 block number where the services expect an L2 block	MEDIUM
M-02	A refund of an unrecognized checkout becomes revenue debt	MEDIUM
L-01	Outflows from spare capital become revenue debt that new fees repay twice	LOW
L-02	An undeliverable referral or manager share blocks the team and depositor shares (V3/V4)	LOW
L-03	V1 and V2 keep an undeliverable share locked in the vault forever	LOW
L-04	Reserved team and depositor shares cannot be redirected after a wallet rotation	LOW
L-05	A signer with an EIP-7702 delegation blocks `resume` until signer rotation	LOW
I-01	Token-level USDC denial stops delivery and migration while it lasts	INFORMATIONAL
I-02	Governance schedules never expire	INFORMATIONAL
I-03	`beginMigration` is irreversible and has no abort path	INFORMATIONAL
I-04	Credit payments store a zero trader, so the self-referral guard does not apply	INFORMATIONAL
I-05	Migration events omit some copied state	INFORMATIONAL
I-06	Predecessor lookups cost more gas with each migration	INFORMATIONAL
I-07	`recognizeRevenue` runs while the vault is paused	INFORMATIONAL
I-08	The V2 `POLICY` constant is dead code	INFORMATIONAL

MEDIUM SEVERITY · 2 FINDINGS

M-01 MEDIUM

The vault records the L1 block number where the services expect an L2 block

On Arbitrum, `block.number` exposes the L1 block number. `completeMigration` and `sweepLateFunds` store that value in `migrationBlock` and `lastSweepBlock`, while the services compare both fields with L2 receipt and event heights.

```
migrationBlock = block.number;
```

```
lastSweepBlock = block.number;
```

The mismatched domains make every L2 receipt look newer than the stored migration marker. Event queries also search the L1 number as an L2 height and find no matching migration event.

IMPACT

The live V4 has no successor, so the defect has no current effect. After the first migration, fees paid to a predecessor are not recognized as revenue, related refunds fail, and migrated share jobs are not routed to the successor.

RECOMMENDATION

Read the L2 `blockNumber` from `MigrationCompleted` and `LateFundsSwept` logs in the services. Future vaults should record `ArbSys.arbBlockNumber()` and tests should use realistic L1 and L2 heights.

```
+interface ArbSys {
+    function arbBlockNumber() external view returns (uint256);
+}
@@
-        migrationBlock = block.number;
+        migrationBlock = ArbSys(address(100)).arbBlockNumber();
@@
-        lastSweepBlock = block.number;
+        lastSweepBlock = ArbSys(address(100)).arbBlockNumber();
```

M-02 MEDIUM

A refund of an unrecognized checkout becomes revenue debt

`refundCheckout` sends the full amount through `_administrativeOutflow`, which charges it to `_spendRevenue`. The services reject a refunded order from revenue recognition, even though the protocol share is returned to the vault before the refund.

```
_spendRevenue(amount);
```

The refund leaves net vault cash unchanged but still reduces `revenueAvailable` or increases `unrecognizedCashSpent`. A direct USDC transfer back to the vault does not clear that accounting debt.

IMPACT

Every ordinary refund removes equal commission capacity from later fees. Affected payouts enter insurance mode and allocate no referral, manager, team, or depositor shares until later revenue repays the false debt.

RECOMMENDATION

Use spare capital above the insurance floor before charging an administrative outflow to recognized revenue.

```
-         _spendRevenue(amount);
+         _spendRevenue(_beyondIdleSurplus(amount));
@@
+     function _beyondIdleSurplus(uint256 amount) private view returns (uint256) {
+         uint256 liquidity = freeLiquidity();
+         uint256 surplus = liquidity > insuranceTarget ? liquidity - insuranceTarget : 0;
+         uint256 idle = surplus > revenueAvailable ? surplus - revenueAvailable : 0;
+         return amount > idle ? amount - idle : 0;
+     }
```

LOW SEVERITY · 5 FINDINGS

L-01 LOW

Outflows from spare capital become revenue debt that new fees repay twice

`_spendRevenue` records any outflow above `revenueAvailable` as `unrecognizedCashSpent`, even when spare capital above the insurance floor funds it. Later recognition first repays that debt, although the live balance already reflects the original outflow.

```
function _spendRevenue(uint256 amount) private {
    if (amount > revenueAvailable) {
        unrecognizedCashSpent += amount - revenueAvailable;
        revenueAvailable = 0;
    } else {
        revenueAvailable -= amount;
    }
}
```

IMPACT

Real fees can produce no distributable revenue after an ordinary Safe capital recovery or a top-up following an insurance payout. Payouts remain in insurance mode until later fees exceed the false debt.

RECOMMENDATION

Apply the idle-surplus helper from M-02 to `recoverUnreserved` so only the portion that crosses into revenue or insurance backing reaches `_spendRevenue`.

```
-         _spendRevenue(amount);
+         _spendRevenue(_beyondIdleSurplus(amount));
```

L-02 LOW

An undeliverable referral or manager share blocks the team and depositor shares (V3/V4)

V3 and V4 require referral and manager shares to settle before team and depositor shares. A fixed wallet that USDC later blacklists, or a beneficiary that never claims an unbound share, leaves that prerequisite permanently unmet.

```
if (r.kind == 1 || r.kind == 3) {
    if ((referral.amount != 0 && !referral.delivered) || (manager.amount != 0 &&
!manager.delivered)) {
        revert InvalidPayment();
    }
}
```

No function cancels or rebinds a share, and migration carries the unresolved obligation into each successor.

IMPACT

The team and depositor shares for that payment cannot be delivered. The Safe can withdraw the backing while paused, but the recorded debt remains and blocks `resume` until the amount is restored.

RECOMMENDATION

Add a timelocked Safe action that cancels or rebinds an undeliverable share while the vault is paused. Preserve an event trail and reduce `reserved` only when the old obligation is explicitly voided.

L-03 LOW

V1 and V2 keep an undeliverable share locked in the vault forever

V1 and V2 bind fixed-wallet shares to the recorded recipient. If USDC blacklists that wallet, delivery always reverts, and these revisions provide neither share cancellation nor a treasury withdrawal that can reach reserved backing.

IMPACT

The Safe cannot recover the affected amount. Other shares for the payment remain deliverable, but the blacklisted recipient's USDC stays reserved in the vault indefinitely.

RECOMMENDATION

Stop creating fixed-wallet shares in V1 and V2. Keep an audited cancel or rebind action in every future vault revision.

L-04 LOW

Reserved team and depositor shares cannot be redirected after a wallet rotation

`_reserve` stores the team and depositor addresses when a payment executes. `changeTeam` and `changeDepositor` affect future payments only, while `deliver` and `importShares` preserve the old recorded recipient.

IMPACT

After a security-driven wallet rotation, outstanding shares remain payable only to the compromised wallet. The signers can withhold release, but the amount stays in `reserved` and may block normal operation.

RECOMMENDATION

Let the Safe rebind an outstanding team or depositor share after the governance delay. An alternative is to resolve those two share kinds from the current configured recipient during delivery.

L-05 LOW

A signer with an EIP-7702 delegation blocks `resume` until signer rotation

`resume` reruns `_checkSigners`, which rejects either signer when its address has code. An EOA with an EIP-7702 delegation has code, so a delegated signer can pause the vault and make the Safe's immediate resume attempt revert.

IMPACT

The vault remains paused until the Safe revokes and rotates the signer through the 1 day governance delay. No funds move, but payment and share processing remain unavailable during recovery.

RECOMMENDATION

Document the revoke, rotate, and resume sequence in the incident runbook. Before a successor deployment, decide whether code checks belong only in signer configuration rather than every `resume` call.

INFORMATIONAL SEVERITY · 8 FINDINGS

I-01 INFORMATIONAL

Token-level USDC denial stops delivery and migration while it lasts

The vaults use native USDC and inherit Circle's pause and blacklist enforcement. A token pause makes delivery, unreserved recovery, and migration transfers revert, while a vault blacklist prevents any contract path from moving its USDC.

IMPACT

Token-level enforcement can suspend payout and migration operations outside Iceberg's control. A blacklisted referral or manager does not itself block migration because migration transfers the vault balance.

RECOMMENDATION

Monitor Circle pause and blacklist events and include payout, recovery, and migration interruption in the incident runbook.

I-02 INFORMATIONAL

Governance schedules never expire

`_consume` accepts a scheduled operation at any time after its 1 day delay. The Safe can cancel it, and state-dependent hashes stop matching after some configuration changes, but no general expiry removes a stale operation.

IMPACT

A matured operation can remain executable longer than operators expect while its hash still matches current state. Only the trusted Safe can execute it.

RECOMMENDATION

Add an expiry window and require the Safe to schedule the operation again after that window closes.

I-03 INFORMATIONAL

`beginMigration` is irreversible and has no abort path

Once `beginMigration` selects a successor, the source vault becomes retired and cannot return to normal operation. The Safe must import every outstanding share and finish migration before activity can continue on the successor.

IMPACT

An abandoned or delayed migration freezes the source until the Safe completes the prepared path. The Safe can still import a share whose recipient is blacklisted because import moves no tokens.

RECOMMENDATION

Treat `beginMigration` as a one-way production step. Prepare the verified successor, full share inventory, service routing, and completion transaction before starting it.

I-04 INFORMATIONAL

Credit payments store a zero trader, so the self-referral guard does not apply

A credit payment has no on-chain trader wallet, so V4 stores `address(0)` in `paymentTraders`. The delivery guard compares a referral recipient with zero and cannot detect that the same person controls both identities.

IMPACT

The on-chain self-referral check does not protect credit payments. A trader can receive a referral share if the off-chain identity checks fail or are bypassed.

RECOMMENDATION

Keep distinct-identity verification mandatory in both signing services. If credit payments later carry a verifiable trader wallet, store it and apply the same delivery check used for wallet payments.

I-05 INFORMATIONAL

Migration events omit some copied state

`ShareImported` reports the payment ID, kind, and amount but omits the copied recipient and beneficiary. `MigrationAccepted` omits the copied team, depositor, revenue, and insurance configuration.

IMPACT

Event-driven monitoring cannot reconstruct or verify all successor state from migration logs alone. Operators must read both vaults after migration.

RECOMMENDATION

Include the copied recipient and beneficiary in share import events and emit a configuration snapshot for team, depositor, revenue, and insurance state.

I-06 INFORMATIONAL

Predecessor lookups cost more gas with each migration

V4 resolves allocation, invalidation, recognized revenue, and refunded checkout state by recursively querying every predecessor. Each migration adds another external call to those lookups and raises the gas used by `execute`.

IMPACT

Gas grows linearly with migration depth. The audit measured about `5.7k` gas per lookup hop and about `11.9k` extra gas for `execute`, with exhaustion near 2,500 hops under the tested environment. The live V4 has no predecessor.

RECOMMENDATION

Track migration depth and transaction gas after every successor deployment. Introduce checkpointed or flattened historical state before the chain becomes operationally significant.

I-07 INFORMATIONAL

`recognizeRevenue` runs while the vault is paused

`recognizeRevenue` checks retirement and migration state but does not check `paused`. Current signers can therefore update revenue accounting while execution and delivery remain blocked.

IMPACT

Pause does not freeze every state transition. No cash moves, recognized credit is bounded by available room, and signer revocation blocks further recognition.

RECOMMENDATION

Document revenue recognition as an intentional pause exception. If pause should freeze all accounting changes, apply the same pause gate used by execution paths.

I-08 INFORMATIONAL

The V2 `POLICY` constant is dead code

V2 exposes `POLICY`, but execution accepts only `RETAINED_POLICY`. V2 also cannot be a V3 or V4 predecessor, so successor policy checks never read the unused value.

IMPACT

The constant has no runtime effect and suggests support for a policy that V2 never accepts.

RECOMMENDATION

Remove the constant from future source revisions or document why it remains in the immutable V2 ABI.